WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Processor Design

Lecture 20: fetch, decode, execute and load; single-cycle, multi-cycle and pipelined designs

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 The four stages

02 Three ways to run the stages

03 Summary

01

The four stages

Building the DX8

The parts we have: the RAM and the register file (Unit 19), the ALU (Unit 18), and registers such as the IP, IR, regA, regB, regOUT and regFLAG (Unit 15).

Two more parts make it run:

- the stepper, which turns on one output at a time, in a fixed order, in step with the clock;
- the controller, which sets the control bits of every other part for the current step.

The question for today

Running one instruction takes several things, one after another: read it, work out what it means, do it, and keep the result. How do we organise that in time?

Fetch, decode, execute, load

Stage	What happens
Fetch	The controller sets RE of the RAM, and WE of the register file and the IR: the instruction is read from memory into the CPU.
Decode	The instruction is examined: is it an ALU or a memory instruction, and which part is responsible for it?
Execute	The instruction is carried out, in the ALU or elsewhere; the results are kept for now in registers such as regOUT and regFLAG.
Load	The kept results are written back where they belong, to the register file or the RAM, if needed. Then the cycle starts again.

> **Remark**
>
> Splitting the work into four stages makes the design easier and modular: we can design and test one stage at a time.

One instruction, four stages

`add:` `regOUT` $\leftarrow$ `regA + regB`

1. **Fetch**: the instruction at the address in the IP comes in from the RAM.
2. **Decode**: $b_{15} = 1$, so the ALU is responsible; the opcode 0001 goes to the ALU.
3. **Execute**: the ALU adds; the sum goes to regOUT, the flags to regFLAG.
4. **Load**: regOUT and regFLAG go back to the register file.

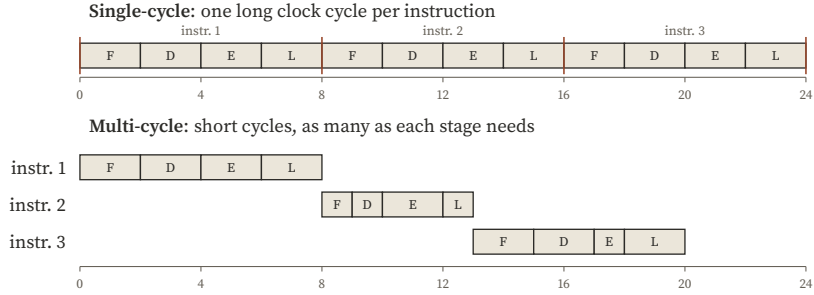
`store:` M`[memAddr]` $\leftarrow$ `regX`

1. **Fetch**: the same as for every instruction.
2. **Decode**: $b_{15} = 0$ and opcode 0001: a `store`.
3. **Execute**: there is nothing for the ALU to do.
4. **Load**: the register's content is written into the RAM, at the address `memAddr` given in the instruction.

02

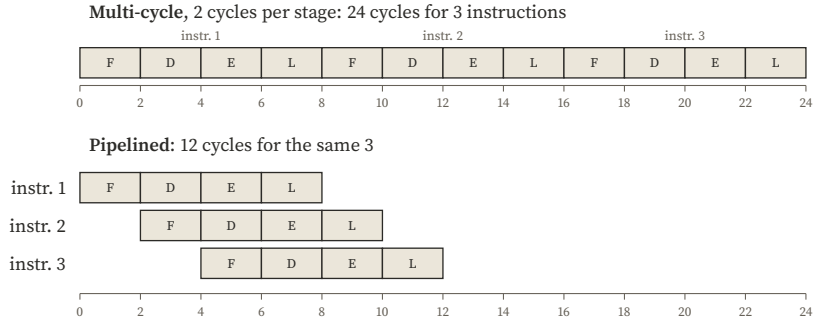
Three ways to run the stages

Single-cycle and multi-cycle



- Single-cycle: the period must fit the slowest instruction. If that takes 100 ns, *every* instruction takes 100 ns, even one that could finish sooner.
- Multi-cycle: here the three instructions take 8, 5 and 7 short cycles: 20 instead of 24. But each instruction still waits for the one before it.

Pipelined



- When instruction 1 moves on to decode, the fetch stage is idle. A pipelined CPU fetches instruction 2 at once: the stages work on different instructions at the same time.
- Each instruction still takes 8 cycles; but one finishes every 2 cycles.

Latency and throughput

Definition (Latency and throughput)

Latency: the time a CPU takes to finish one instruction. *Throughput*: how many instructions it finishes per unit of time.

- Single-cycle to multi-cycle: shorter latency, for instructions that need fewer cycles.
- Multi-cycle to pipelined: the same latency, more throughput.

	Multi-cycle	Pipelined
Latency	8	8
3 instructions	24	12
Per cycle	$\frac{3}{24} = 0.125$	$\frac{3}{12} = 0.25$
n instructions	$8n$	$2(n+3)$

4 stages of 2 cycles each. For long programs the pipeline finishes one instruction every 2 cycles: 4 times the throughput.

The catch

- A pipeline needs registers between its stages, so that each stage can hold its own instruction: the “register in the middle of a long path” of `(Unit 14)`.
- An instruction may need a result that the instruction just before it has not written back yet.
- After a jump, the instructions already fetched behind it may be the wrong ones.
- Handling these cases is what makes pipelined CPUs the hardest of the three to design.

> **Remark**
>
> These problems are called *hazards*. They are beyond this module: the DX8 is not pipelined, so it never meets them.

The DX8's choice

Design	To build	Performance
Single-cycle	simplest	lowest
Multi-cycle	moderate	moderate
Pipelined	hardest	highest

- The book chooses multi-cycle for the DX8: a trade-off between performance and simplicity.
- Each stage gets at most 2 clock cycles, or *steps*: fetch is steps 1 and 2, decode 3 and 4, execute 5 and 6, load 7 and 8. Then back to step 1.
- The stepper counts the steps `(Unit 21)`.

Try it

1. In a single-cycle CPU, the slowest instruction takes 100 ns. What is the fastest clock, and how long does the quickest instruction take?
   10 MHz; 100 ns, like every other instruction.
2. The DX8 takes 8 cycles per instruction. With a 1 MHz clock, how many instructions does it finish per second?
   $10^6 / 8 = 125\,000$.
3. 4 stages of 2 cycles each. How many cycles do 10 instructions take, multi-cycle and pipelined?
   $8 \cdot 10 = 80$, and $2 \cdot (10 + 3) = 26$.
4. Does pipelining make one instruction finish sooner?
   No: its latency stays 8 cycles. More instructions finish per second.

03

Summary

Summary

- Every instruction goes through four stages: fetch, decode, execute and load.
- Single-cycle: all four in one long clock cycle, set by the slowest instruction.
- Multi-cycle: shorter cycles; each stage takes the cycles it needs, one instruction at a time.
- Pipelined: the stages overlap, working on different instructions; the same latency, more throughput.
- The DX8 is multi-cycle: 8 steps per instruction, 2 for each stage.

Before the next lecture

- Read the beginning of Section 6.6 and Section 6.6.1 of the book.
- Draw the time line of 5 instructions on the multi-cycle DX8 and on a pipelined DX8, and count the cycles.
- Read Sections 6.6.2 and 6.6.3.

> **Next lecture** (Unit 21)
>
> The stepper, the data path and the controller.